

리눅스 사용자 · 그룹 · 권한 관리

인프라 엔지니어 실무 정리 노트 — 계정, 그룹, 환경 프로파일, 리소스 제한, sudo, root 접근

목차

1. 로컬 사용자 계정 관리
2. 그룹과 그룹 멤버십
3. 시스템 전역 환경 프로파일
4. 사용자 환경 템플릿 (/etc/skel)
5. 사용자 리소스 제한
6. sudo 권한 관리
7. root 계정 접근 통제
8. 부록 — 배포판 차이 / 실습 체크리스트 / 면접 대비 질문

1. 로컬 사용자 계정 관리

1.1 왜 사람마다 계정을 따로 주는가

서버에 접속하는 사람이 여럿이면 계정을 공유하지 않고 각자에게 별도 계정을 발급하는 것이 원칙입니다. 이유는 세 가지로 정리됩니다.

- **격리** — 각자의 홈 디렉터리와 개인 설정이 퍼미션으로 분리됩니다.
- **최소 권한** — 업무에 필요한 만큼만 권한을 주면, 실수로 잘못된 명령을 실행해도 피해 범위가 그 계정 안에 갇힙니다.
- **감사 추적** — 로그에 누가 무엇을 했는지가 계정 단위로 남습니다. 공용 계정을 쓰면 사고가 났을 때 책임 추적이 불가능해집니다. 실무에서 이게 가장 중요합니다.

1.2 계정 생성 — adduser와 useradd

리눅스에는 계정을 만드는 명령이 두 가지 계열로 존재합니다. 이 둘의 차이를 아는 것이 첫 관문입니다.

구분	useradd	adduser
성격	저수준 유틸리티. 모든 배포판 공통	Debian/Ubuntu의 대화형 래퍼 스크립트
동작	옵션을 안 주면 홈 디렉터리도 안 만들(배포판 기본값에 따라 다름)	홈 생성·비밀번호 설정·부가정보 입력까지 한 번에 진행
RHEL 계열	표준	useradd를 가리키는 심볼릭 링크인 경우가 많음
스크립트 자동화	적합 (비대화형)	부적합 (프롬프트가 뜸)

면접에서 자주 나오는 포인트입니다. “**Ubuntu에서는 adduser, RHEL/Rocky에서는 useradd**” 정도로 외우면 절반만 맞습니다. 정확히는 *useradd가 표준 저수준 명령이고, adduser는 데비안 계열이 제공하는 편의 래퍼이며, Ansible 같은 자동화 도구는 항상 useradd 계열 동작(ansible.builtin.user)을 씁니다.*

대화형 생성 (Debian/Ubuntu)

```
sudo adduser jihoon
```

실행하면 비밀번호를 두 번 입력받고, 이름·전화번호 같은 부가 정보를 물어봅니다. 부가 정보는 대부분 그냥 Enter로 건너뛰고 마지막에 **y** 를 눌러 확정합니다. 이 한 줄이 실제로 하는 일은 다음과 같습니다.

- `jihoon` 계정이 생성되고 `/etc/passwd` 에 항목이 추가됨
- 같은 이름의 그룹 `jihoon` 이 만들어지고 이 계정의 **주 그룹(primary group)**으로 지정됨 — 이를 UPG(User Private Group) 방식이라고 합니다

- 홈 디렉터리 `/home/jihoon` 생성
- 기본 로그인 셸이 `/bin/bash` 로 지정됨
- `/etc/skel` 안의 파일이 홈 디렉터리로 복사됨 (4장 참고)
- 계정 만료일은 설정되지 않음(무기한)

비대화형 생성 (자동화 / RHEL 계열)

```
sudo useradd -m -s /bin/bash -c "SRE Team - Jihoon" jihoon
sudo passwd jihoon
```

```
# -m : 홈 디렉터리 생성 (RHEL은 기본 생성이지만 명시하는 습관을 들일 것)
# -s : 로그인 셸 지정
# -c : GECOS 필드(설명). 계정 용도를 적어두면 인수인계가 편함
```

1.3 비밀번호 설정

계정만 만들면 비밀번호가 없어 로그인할 수 없습니다(정확히는 잠긴 상태). 비밀번호를 부여합니다.

```
sudo passwd jihoon
```

스크립트에서 비대화형으로 처리해야 한다면 `chpasswd` 를 씁니다. 다만 평문 비밀번호가 셸 히스토리에 남으므로 실무에서는 권장되지 않으며, 초기 비밀번호를 강제 변경하도록 묶어서 사용합니다.

```
echo 'jihoon:InitPass!2026' | sudo chpasswd
sudo chage -d 0 jihoon # 최초 로그인 시 비밀번호 변경 강제
```

1.4 기본값과 다르게 만들기

```
sudo adduser --shell /usr/sbin/nologin --home /srv/data/jihoon jihoon
```

셸만 바꾸고 홈 위치는 기본값을 쓰고 싶다면 해당 옵션만 주면 됩니다. 홈 디렉터를 `/home` 밖에 두는 경우는 별도 볼륨(NFS, 별도 LV)에 사용자 데이터를 두는 환경에서 흔합니다.

1.5 UID / GID 체계

계정 정보는 `/etc/passwd` 에 한 줄씩 저장됩니다. 콜론으로 구분된 7개 필드입니다.

```
jihoon:x:1001:1001:SRE Team:/home/jihoon:/bin/bash
|   |   |   |   |   |   |
|   |   |   |   |   |   +-- 로그인 셸
|   |   |   |   |   +----- 홈 디렉터리
|   |   |   +----- GECOS (설명)
|   |   +----- 주 그룹 GID
|   +----- UID
| +----- 비밀번호 자리(x = shadow 사용)
+----- 사용자명
```

두 번째 필드의 `x` 는 “실제 해시는 `/etc/shadow` 에 있다”는 표시입니다. `/etc/passwd` 는 모든 사용자가 읽을 수 있어야 하지만(UID↔이름 변환 때문), 비밀번호 해시까지 노출되면 안 되므로 분리한 구조입니다. `/etc/shadow` 는 root만 읽을 수 있습니다.

UID 대역 관례

UID	용도
0	root
1 ~ 999	시스템 계정 (데몬용). 배포판별로 경계가 조금 다름

1000 이상	일반 사용자 계정. 새 계정은 1000부터 순차 부여
---------	-------------------------------

UID를 직접 지정하려면:

```
sudo useradd -u 1500 -m minseo
```

여러 서버에 걸쳐 같은 사람이 같은 UID를 갖도록 맞추는 것은 NFS 공유 환경에서 특히 중요합니다. UID가 서버마다 다르면 NFS 마운트에서 소유권이 엉킵니다. 실무에서는 이 문제 때문에 LDAP/SSSD 같은 중앙 인증으로 넘어갑니다.

소유권을 숫자로 확인하기

```
ls -l /home/      # 이름으로 표시
ls -ln /home/     # UID/GID 숫자로 표시
```

-n 옵션이 유용한 경우가 있습니다. 이름이 안 뜨고 숫자만 보인다면, 해당 UID에 대응하는 계정이 *그 시스템에* 없다는 뜻입니다. 컨테이너 볼륨이나 다른 서버에서 옮겨온 파일에서 자주 마주칩니다.

1.6 내가 누구인지 확인하기

```
whoami      # 현재 사용자명만
id          # UID, GID, 소속 그룹 전부
id jihoon   # 특정 사용자 정보
groups      # 소속 그룹 목록
```

1.7 시스템 계정

사람이 아니라 데몬이 쓰는 계정입니다. nginx, postgres, prometheus 같은 서비스는 root가 아닌 전용 계정으로 돌리는 것이 기본입니다. 서비스가 침해당해도 피해가 그 계정 권한으로 제한되기 때문입니다.

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appsvc
```

- `--system` — 1000 미만의 UID를 부여
- `--no-create-home` — 홈 디렉터리 불필요
- `--shell /usr/sbin/nologin` — **대화형 로그인 차단**. 실무에서 이걸 빠뜨리면 안 됩니다

`/usr/sbin/nologin` 과 `/bin/false` 는 둘 다 로그인을 막지만, 전자는 “이 계정은 사용할 수 없습니다” 메시지를 보여주고 후자는 조용히 실패합니다. 데비안 계열은 `/usr/sbin/nologin`, RHEL 계열은 `/sbin/nologin` 경로를 씁니다.

1.8 계정 수정 — usermod

작업	명령
홈 디렉터리 변경 + 기존 파일 이동	<code>sudo usermod -d /srv/jihoon -m jihoon</code>
사용자명 변경	<code>sudo usermod -l jihoon2 jihoon</code>
로그인 셸 변경	<code>sudo usermod -s /bin/zsh jihoon</code>
계정 잠금	<code>sudo usermod -L jihoon</code>
계정 잠금 해제	<code>sudo usermod -U jihoon</code>
계정 만료일 지정	<code>sudo usermod -e 2026-12-31 jihoon</code>
만료일 제거	<code>sudo usermod -e "" jihoon</code>

`-m` (`--move-home`)을 빼먹으면 `/etc/passwd` 의 홈 경로만 바뀌고 실제 파일은 옛 위치에 남습니다. 사용자가 로그인하면 빈 홈에 떨어지고 기존 파일이 사라진 것처럼 보입니다. 반드시 `-d` 와 `-m` 을 같이 씁니다.

계정 잠금(-L)의 실제 동작을 오해하기 쉽습니다. `usermod -L` 은 `/etc/shadow` 의 비밀번호 해시 앞에 `!` 를 붙여 어떤 비밀번호와도 매칭되지 않게 만드는 것뿐입니다. 즉 **비밀번호 로그인만** 막힙니다. SSH 공개키가 이미 등록되어 있으면 그 사람은 여전히 로그인할 수 있습니다.

퇴사자 계정을 차단할 때는 이것만으로 부족합니다. 최소한 다음을 함께 처리해야 합니다: `usermod -L` + `usermod -s /usr/sbin/nologin` + `~/.ssh/authorized_keys` 제거 + `chage -E 0` (계정 즉시 만료) + 실행 중인 세션 종료(`kill -u`).

1.9 비밀번호 정책 — chage

`chage` 는 “change age”의 줄임말입니다. **계정 만료**와 **비밀번호 만료**는 다른 개념이라는 점을 먼저 구분해야 합니다.

	계정 만료 (usermod -e / chage -E)	비밀번호 만료 (chage -d, -M)
효과	로그인 자체가 완전히 차단됨	로그인은 되지만 즉시 비밀번호 변경을 요구
용도	계약직 종료일, 퇴사 처리	주기적 비밀번호 교체 정책

작업	명령
다음 로그인 시 비밀번호 변경 강제	<code>sudo chage -d 0 jihoon</code>
위 설정 취소	<code>sudo chage -d -1 jihoon</code>
90일마다 변경 강제	<code>sudo chage -M 90 jihoon</code>
비밀번호 무기한	<code>sudo chage -M -1 jihoon</code>
만료 7일 전 경고	<code>sudo chage -W 7 jihoon</code>
현재 설정 조회	<code>sudo chage -l jihoon</code>

신규 계정 발급 표준 절차로 `chage -d 0` 을 넣어두면, 관리자가 정한 초기 비밀번호를 사용자가 반드시 자기 것으로 바꾸게 되어 초기 비밀번호가 그대로 남아 있는 상황을 막을 수 있습니다.

1.10 계정 삭제

```
sudo userdel jihoon          # 계정만 삭제, 홈 디렉터리는 남김
sudo userdel -r jihoon       # 홈 디렉터리와 메일 스펠까지 삭제

# Debian/Ubuntu의 래퍼
sudo deluser jihoon
sudo deluser --remove-home jihoon
```

실무에서는 퇴사자 계정을 **즉시 삭제하지 않습니다**. 이유는 두 가지입니다. (1) 홈 디렉터리에 인수인계가 안 된 업무 데이터가 남아 있을 수 있습니다. (2) 계정을 지우면 UID가 회수되어 나중에 다른 사람에게 재할당될 수 있고, 그러면 그 사람이 **퇴사자가 남긴 파일의 소유자**가 되어버립니다. 표준 절차는 **잠금 → 유예 기간(보통 30~90일) → 데이터 백업 → 삭제**입니다.

2. 그룹과 그룹 멤버십

2.1 그룹이 해결하는 문제

개발팀 3명이 `/srv/project` 아래 파일을 함께 수정해야 한다고 합니다. 파일마다 세 사람에게 각각 권한을 주는 방식은 사람이 늘어날수록 관리가 무너집니다. 그룹을 쓰면 이렇게 정리됩니다.

1. `developers` 그룹을 만든다
2. 세 사람을 그 그룹에 넣는다
3. 디렉터리의 그룹 소유자를 `developers` 로 바꾸고 그룹에 `rw` 권한을 준다

이후로는 **사람이 아니라 그룹 멤버십만 조작**하면 됩니다. 신규 입사자는 그룹에 추가, 팀 이동자는 그룹에서 제거. 파일 퍼미션은 건드릴 일이 없습니다. 이것이 그룹을 “역할(role)”로 쓰는 방식이고, 리눅스 권한 설계의 기본 패턴입니다.

2.2 그룹은 권한 그 자체이기도 하다

일부 그룹은 소속만으로 특별한 권한을 부여합니다. 이건 파일 퍼미션과는 별개의 메커니즘입니다.

그룹	부여되는 권한
<code>sudo</code> (Debian/Ubuntu)	<code>sudo</code> 로 사실상 모든 명령 실행 가능
<code>wheel</code> (RHEL/Rocky)	위와 동일. 배포판만 다름
<code>docker</code>	도커 컨테이너 관리
<code>adm</code>	<code>/var/log</code> 로그 파일 읽기
<code>libvirt</code> / <code>kvm</code>	가상 머신 관리

`docker` 그룹은 사실상 `root` 권한과 같습니다. 도커 데몬이 `root`로 동작하므로, 컨테이너에 호스트의 `/` 를 마운트하면 그대로 호스트 전체를 장악할 수 있습니다. “`sudo` 권한은 안 줬는데 `docker` 그룹은 넣어줬다”는 것은 보안상 의미가 없습니다. 면접에서 한 번쯤 물어보는 포인트입니다.

2.3 주 그룹 vs 보조 그룹

사용자는 여러 그룹에 속할 수 있는데, 그중 하나가 **주 그룹(primary/login group)**이고 나머지는 **보조 그룹(secondary/supplementary group)**입니다. 차이는 다음 두 가지에서 드러납니다.

- 사용자가 **새 파일을 만들면** 그 파일의 그룹 소유자는 **주 그룹**이 됩니다
- 사용자가 실행한 **프로세스**는 그 계정과 주 그룹의 권한으로 동작합니다

보조 그룹은 “접근 가능 여부”를 판정할 때만 쓰입니다. 즉 보조 그룹은 *읽고 쓸 수 있게* 해주지만, 새로 만드는 파일에 자동으로 붙지는 않습니다. 이 차이가 협업 디렉터리에서 문제를 일으킵니다(2.7절 참고).

2.4 그룹 생성과 멤버십 관리

```
sudo groupadd developers
sudo groupadd -g 5000 developers    # GID 직접 지정
```

사용자를 그룹에 넣는 방법은 두 가지입니다.

방법 1 — `gpasswd` (권장)

```
sudo gpasswd -a jihoon developers    # 추가 (--add)
sudo gpasswd -d jihoon developers    # 제거 (--delete)
```

방법 2 — `usermod`

```
sudo usermod -aG developers jihoon    # 보조 그룹에 추가
```

```
sudo usermod -g developers jihoon # 주 그룹을 변경
```

가장 자주 나는 사고입니다. `usermod -G developers jihoon` 처럼 `-a` 없이 대문자 `-G` 만 쓰면, 기존 보조 그룹이 전부 날아가고 developers만 남습니다. 관리자 본인에게 이걸 실행하면 `sudo` 그룹에서 빠져 서버에서 잠기는 일이 실제로 발생합니다.

암기법: **-aG는 항상 붙여서 쓴다.** `-a` 는 append입니다. 소문자 `-g` 는 주 그룹, 대문자 `-G` 는 보조 그룹입니다.

인자 순서가 명령마다 반대입니다. `gpasswd -a 사용자 그룹` / `usermod -aG 그룹 사용자`. 헷갈리면 `gpasswd --help` 의 `-a, --add USER` 표기를 보고 확인하면 됩니다.

2.5 확인

```
groups jihoon
# jihoon : jihoon developers docker
#          ^^^^^^ 콜론 바로 뒤 첫 번째가 주 그룹, 나머지는 보조 그룹

id jihoon
getent group developers # 그룹에 속한 멤버 목록
```

`getent` 는 로컬 파일(`/etc/group`)뿐 아니라 LDAP·SSSD 같은 중앙 인증 소스까지 함께 조회합니다. `cat /etc/group` 보다 `getent group` 을 쓰는 습관을 들이면 중앙 인증 환경에서도 그대로 통합니다.

2.6 그룹 수정·삭제

```
sudo groupmod -n platform developers # 이름 변경 (신규명 → 기존명 순서)
sudo groupmod -g 5100 platform       # GID 변경
sudo groupdel platform               # 삭제
```

삭제하려는 그룹이 누군가의 **주 그룹**이면 다음 오류로 거부됩니다.

```
groupdel: cannot remove the primary group of user 'jihoon'
```

해당 사용자의 주 그룹을 먼저 원래대로 되돌린 뒤 삭제합니다.

```
sudo usermod -g jihoon jihoon
sudo groupdel platform
```

보조 그룹인 경우에는 멤버를 먼저 빼지 않아도 그냥 삭제됩니다.

그룹을 지우면 그 GID로 소유된 파일들은 **소유자 없는 숫자 GID**로 남습니다. `ls -l` 에 이름 대신 숫자가 보이면 이 상황입니다. 삭제 전에 `find / -gid <GID> 2>/dev/null` 로 해당 그룹 소유 파일을 확인하는 것이 안전합니다.

2.7 실전 — 협업 디렉터리 제대로 만들기

2.3절에서 언급한 문제를 해결하는 표준 레시피입니다. 그룹만 만들어서는 부족합니다.

```
sudo groupadd developers
sudo gpasswd -a jihoon developers
sudo gpasswd -a minseo developers

sudo mkdir -p /srv/project
sudo chgrp developers /srv/project
sudo chmod 2775 /srv/project
#          ^ setgid 비트
```

`chmod 2775` 의 맨 앞 `2` 가 **setgid** 비트입니다. 디렉터리에 setgid를 걸면 그 안에서 만들어지는 모든 파일이 만든 사람의 주 그룹이 아니라 디렉터리의 그룹을 상속합니다. 이게 없으면 jihoon이 만든 파일은 그룹이 `jihoon` 이 되어 minseo가 못 읽습니다.

여기에 umask까지 맞춰주면 완성입니다. 기본 umask 022 는 그룹 쓰기 권한을 깎아버리므로, 협업 환경에서는 002 로 두어야 그룹 쓰기가 유지됩니다.

```
umask 002
```



3. 시스템 전역 환경 프로파일

3.1 환경 변수란

셸과 프로그램이 참조하는 키=값 설정입니다. 현재 세션의 환경 변수는 다음으로 확인합니다.

```
printenv      # 또는
env
echo $HOME    # 개별 변수 조회
```

변수 앞에 \$ 를 붙이면 그 자리에 값이 치환됩니다. 스크립트에서 이 성질이 중요합니다.

```
touch $HOME/report.log
```

jihoon이 실행하면 /home/jihoon/report.log , minseo가 실행하면 /home/minseo/report.log 가 만들어집니다. 하나의 스크립트가 사용자마다 다르게 동작하는 것이 아니라, *각자의 환경에 맞게 자동으로* 조정되는 것입니다.

자주 쓰는 변수

변수	의미
PATH	명령을 찾을 디렉터리 목록. 콜론으로 구분
HOME	현재 사용자의 홈 디렉터리
USER	현재 사용자명
SHELL	로그인 셸 경로
PWD	현재 작업 디렉터리
LANG / LC_ALL	로케일. ko_KR.UTF-8 등
HISTSIZE	셸이 기억할 명령 히스토리 개수
EDITOR	기본 편집기. visudo 등이 참조

3.2 셸 변수 vs 환경 변수 — export의 의미

이 구분을 모르면 “변수를 설정했는데 스크립트에서 안 보인다”는 문제를 겪습니다.

```
MYVAR=hello      # 셸 변수 — 현재 셸에서만 보임
export MYVAR=hello # 환경 변수 — 자식 프로세스까지 상속됨
```

스크립트를 실행하면 새 셸(자식 프로세스)이 뜨므로, export 하지 않은 변수는 그 안에서 보이지 않습니다.

3.3 설정 파일 — 어디에 무엇을 쓰는가

이 표가 3장의 핵심입니다.

파일	적용 범위	용도
/etc/environment	전체 사용자	변수 정의 전용. 셸 스크립트가 아님. KEY=value 형식만 가능하고 export 나 변수 확장, 조건문을 쓸 수 없음
/etc/profile	전체 사용자	로그인 셸 시작 시 실행. 배포판이 관리하므로 직접 수정보다 아래 방식 권장
/etc/profile.d/*.sh	전체 사용자	전역 설정을 추가할 때 권장되는 위치. 셸 스크립트라 조건문·명령 실행 가능
~/.bash_profile	해당 사용자	로그인 셸에서만 실행 (SSH 접속 등)
~/.bashrc	해당 사용자	대화형 비로그인 셸에서 실행 (터미널 새 탭 등)

왜 `/etc/profile`을 직접 고치지 말라고 하는가. 패키지 업데이트 시 배포판이 이 파일을 덮어쓸 수 있고, 그러면 수정 내용이 사라집니다. `/etc/profile.d/`에 별도 `.sh` 파일을 두면 업데이트에 영향받지 않고, 설정을 파일 단위로 추가·제거할 수 있어 관리도 쉽습니다. 이 “드롭인 디렉터리” 패턴은 리눅스 전반에서 반복됩니다 — `/etc/sudoers.d/`, `/etc/security/limits.d/`, `/etc/systemd/system/*.d/` 모두 같은 사상입니다.

3.4 전역 변수 추가하기

```
sudo vim /etc/environment
```

```
DEPLOY_ENV=production
JAVA_HOME=/usr/lib/jvm/java-17-openjdk
```

저장 후 로그아웃했다 다시 로그인하면 적용됩니다. 확인:

```
echo $DEPLOY_ENV
```

3.5 로그인 시 명령 실행하기

`/etc/environment`는 변수만 정의할 수 있습니다. 명령을 실행하거나 조건 분기가 필요하면 `/etc/profile.d/`를 씁니다.

```
sudo vim /etc/profile.d/motd-notice.sh
```

```
echo "마지막 로그인: $(date '+%Y-%m-%d %H:%M:%S') " > "$HOME/.lastlogin"
```

여기 있는 `.sh` 파일에는 `#!/bin/bash` 세행이 필요 없습니다. 별도 프로세스로 실행되는 게 아니라 현재 셸이 직접 읽어들이기(source) 때문입니다.

`/etc/profile.d/`의 스크립트는 **모든 사용자의 로그인 경로에 끼어듭니다**. 여기서 무거운 명령을 돌리면 전체 사용자의 로그인이 느려지고, 문법 오류를 내면 최악의 경우 아무도 로그인하지 못하게 됩니다. 수정한 뒤에는 **현재 SSH 세션을 끊지 말고** 새 창으로 접속해 정상 동작을 먼저 확인하세요. 이걸 `sudoers`를 편집할 때와 동일한 안전 수칙입니다.

3.6 로그인 셸과 비로그인 셸

“`.bashrc`에 넣었는데 왜 SSH 접속 시 적용이 안 되나요?”가 자주 나오는 질문입니다.

상황	읽는 파일
SSH 접속, <code>su -</code> , 콘솔 로그인 (로그인 셸)	<code>/etc/profile</code> → <code>/etc/profile.d/*</code> → <code>~/.bash_profile</code>
터미널 새 탭, <code>bash</code> 실행 (대화형 비로그인 셸)	<code>/etc/bash.bashrc</code> → <code>~/.bashrc</code>
스크립트 실행 (비대화형)	둘 다 안 읽음. <code>BASH_ENV</code> 만 참조

그래서 대부분의 배포판은 `~/.bash_profile` 안에서 `~/.bashrc`를 불러오도록 해 두 경로를 통일시킵니다.

```
[ -f ~/.bashrc ] && . ~/.bashrc
```

결론적으로 **개인 설정은 `~/.bashrc`에 넣고, `~/.bash_profile`에서는 그것을 source하는 것**이 표준 구성입니다.

4. 사용자 환경 템플릿 — /etc/skel

4.1 동작 원리

새 계정을 만들 때 /etc/skel (skeleton, 골격) 디렉터리의 내용이 그대로 새 홈 디렉터리로 복사됩니다. 즉 **신규 사용자 홈의 초기 상태를 정의하는 템플릿**입니다.

```
ls -a /etc/skel/  
# . . . .bash_logout .bash_profile .bashrc
```

-a 옵션이 필요합니다. 점(.)으로 시작하는 파일은 숨김 파일이라 기본 출력에 안 나옵니다.

4.2 활용 1 — 공지 파일 배포

```
sudo vim /etc/skel/README.txt
```

여기에 서버 이용 정책을 적어두면, 앞으로 생성되는 모든 계정의 홈에 이 파일이 들어갑니다. 기존 사용자에게는 소급 적용되지 않는다는 점만 기억하면 됩니다.

```
sudo useradd -m trinity  
sudo ls -a /home/trinity/  
# .bash_logout .bash_profile .bashrc README.txt
```

4.3 활용 2 — PATH 등 기본 환경 통일

사내 도구를 /opt/tools/bin 에 두었다면, 사용자가 전체 경로를 입력하지 않고 명령 이름만으로 실행할 수 있게 PATH에 추가해야 합니다.

```
sudo vim /etc/skel/.bashrc
```

```
PATH="$HOME/.local/bin:$HOME/bin:/opt/tools/bin:$PATH"
```

PATH 편집 시 주의점 두 가지.

- (1) 구분자는 콜론(:)입니다. 앞뒤로 콜론이 정확히 하나씩 있는지 확인하세요. 빈 항목이 생기면 그건 “현재 디렉터리”로 해석되어 보안 위험이 됩니다.
- (2) 새 경로는 \$PATH 앞에 두어야 우선 적용됩니다. 반대로 시스템 기본 명령을 덮어쓰면 안 되는 상황이라면 \$PATH 뒤에 붙입니다.

이미 만들어진 계정에도 적용하려면 그 사용자의 ~/.bashrc 를 직접 수정하거나, 전체 적용이라면 3장의 /etc/profile.d/ 를 쓰는 편이 낫습니다.

적용 대상	수정할 위치
앞으로 만들 계정만	/etc/skel/.bashrc
특정 기존 사용자만	/home/사용자/.bashrc
기존 + 신규 전체	/etc/profile.d/사내설정.sh

4.4 실무 활용 예

- 사내 표준 .vimrc , .gitconfig 템플릿 배포
- .ssh/ 디렉터리를 700 퍼미션으로 미리 생성해두기
- 사내 CA 인증서 경로, 프록시 설정 환경 변수 주입
- 기본 umask 002 설정 (협업 서버)

/etc/skel 에 넣은 파일의 퍼미션은 그대로 복사됩니다. 실수로 비밀 정보(토큰, 키)를 넣으면 이후 생성되는 모든 계정에 배포됩니다. 넣기 전에 ls -la /etc/skel/ 로 반드시 확인하세요.

5. 사용자 리소스 제한

5.1 왜 필요한가

다중 사용자 서버에서 한 사람이 CPU를 80% 점유하거나, 잘못 짠 스크립트가 프로세스를 무한 생성하면 서버 전체가 마비됩니다. 이를 막기 위해 계정별 자원 상한을 겁니다. 대표적인 사고가 **fork bomb**이고, 이를 막는 것이 `nproc` 제한입니다.

5.2 설정 파일과 문법

```
sudo vim /etc/security/limits.conf
# 또는 (권장) 드롭인 방식
sudo vim /etc/security/limits.d/90-team.conf
```

문법은 네 개 필드입니다.

```
<domain> <type> <item> <value>
```

① domain — 누구에게

표기	대상	예시
사용자명	해당 사용자	<code>jihoon hard nproc 200</code>
@그룹명	그룹 전체	<code>@developers soft nproc 100</code>
*	기본값 (개별 지정이 없는 모든 사용자)	<code>* soft nofile 4096</code>

* 는 기본값 역할입니다. 개별 사용자 항목이 따로 있으면 그쪽이 우선합니다. 예를 들어 전체에 100, jihoon에게 200을 걸어두면 jihoon만 200이 적용됩니다.

② type — 어떤 성격의 제한인가

type	의미
hard	절대 상한. 일반 사용자가 넘을 수 없음. root만 변경 가능
soft	초기값. 로그인 시 적용되는 값. 사용자가 hard 한도까지 스스로 올릴 수 있음
-	hard와 soft를 동시에 같은 값으로 설정

```
jihoon hard nproc 200
jihoon soft nproc 100
```

이 설정이면 jihoon은 로그인 시 100개로 시작하고, 필요할 때 `ulimit -u 150` 처럼 200까지 스스로 올릴 수 있습니다. 평소에는 낮게 유지하고 필요할 때만 여유를 주는 구성입니다.

③ item — 무엇을 제한하는가

item	의미	단위
nproc	최대 프로세스 수	개
nofile	최대 열린 파일 디스크립터 수	개
fsize	생성 가능한 최대 파일 크기	KB
cpu	누적 CPU 시간	분
as	가상 메모리 총량	KB
memlock	잠글 수 있는 메모리	KB

maxlogins	동시 로그인 세션 수	개
core	코어 덤프 파일 크기	KB

cpu 는 “경과 시간”이 아니라 **CPU를 실제로 점유한 누적 시간**입니다. 프로세스가 3시간 떠 있어도 대부분 유휴 상태였다면 CPU 시간은 2초일 수 있습니다. 한 코어를 100% 쓰면 1초당 1초, 50%만 쓰면 1초당 0.5초가 소모됩니다.

실무에서 압도적으로 자주 만지는 항목은 **nofile** 입니다. Nginx, Elasticsearch, PostgreSQL, Kafka 같은 서비스는 동시 커넥션마다 파일 디스크립터를 쓰기 때문에 기본값 1024로는 부족합니다. Too many open files 오류를 보면 심중팔구 이 값입니다.

5.3 현재 세션의 제한 확인·변경 — ulimit

```
ulimit -a      # 전체 확인
ulimit -u      # 최대 프로세스 수 (soft)
ulimit -Hu     # hard 값 확인
ulimit -n 8192 # 현재 세션의 파일 디스크립터 한도 변경
```

ulimit -a 출력의 괄호 안 문자가 곧 해당 항목의 옵션 문자입니다. 예를 들어 max user processes (-u) 라면 ulimit -u 로 조회·변경합니다.

규칙은 이렇습니다. 일반 사용자는 **soft 값을 hard 한도까지 올릴 수 있고, 내리는 것은 자유**입니다. 단 한 번 내린 값은 같은 세션에서 다시 올릴 수 없습니다. hard 값 자체를 올리는 것은 root만 가능합니다.

5.4 검증 실습

```
sudo vim /etc/security/limits.d/99-test.conf
```

```
trinity - nproc 3
```

```
sudo -iu trinity      # -i: 로그인 셸로 전환, -u: 대상 사용자

ps | less             # bash + ps + less = 3개. 정상 동작
ls -a | grep bash | less
# bash: fork: retry: Resource temporarily unavailable
# → 4개째에서 차단됨. 제한이 정상 적용된 것
```

limits.conf 는 **PAM(pam_limits.so)을 거치는 로그인 세션에만 적용**됩니다. 따라서 systemd로 기동되는 서비스에는 적용되지 않습니다. 데몬의 한도를 올리려면 유닛 파일에 직접 지정해야 합니다.

```
sudo systemctl edit nginx
```

```
[Service]
LimitNOFILE=65535
```

“limits.conf를 고쳤는데 서비스에 적용이 안 된다”는 문의의 원인 대부분이 이것입니다. 실무 감각을 보여줄 수 있는 포인트라 면접에서도 유용합니다.

6. sudo 권한 관리

6.1 sudo가 하는 일

시스템의 중요한 영역은 root(superuser)만 변경할 수 있습니다. `sudo` 를 앞에 붙이면 그 명령만 root 권한으로 실행됩니다. root로 계속 로그인해 있는 것보다 훨씬 안전한 이유는 세 가지입니다.

- 필요한 명령에만 한정해서 권한이 상승됨
- 누가 언제 무엇을 실행했는지 로그(`/var/log/auth.log` , `/var/log/secure`)에 남음
- root 비밀번호를 공유하지 않아도 됨 — 각자 자기 비밀번호를 입력

6.2 가장 간단한 방법 — 그룹에 추가

```
# Debian / Ubuntu
sudo gpasswd -a trinity sudo

# RHEL / Rocky / CentOS
sudo gpasswd -a trinity wheel
```

이 그룹에 속하면 사실상 모든 명령을 실행할 수 있습니다. 간편하지만 **전부 아니면 전무**라는 것이 단점입니다. 세밀한 통제가 필요하면 정책을 직접 정의합니다.

6.3 sudoers 편집 — 반드시 visudo로

```
sudo visudo          # /etc/sudoers 편집
sudo visudo -f /etc/sudoers.d/90-deploy  # 드롭인 파일 편집 (권장)
```

/etc/sudoers 를 **vim**으로 직접 열지 마세요. 문법 오류가 있는 상태로 저장하면 sudo 자체가 동작하지 않게 되고, root 로그인이 막혀 있다면 서버에서 완전히 잠깁니다. 복구하려면 싱글 유저 모드로 부팅하거나 복구 콘솔이 필요합니다.

`visudo` 는 저장 시 문법을 검사하고, 오류가 있으면 다시 편집할 기회를 줍니다. 이 안전장치 때문에 존재하는 명령입니다.

개별 정책은 `/etc/sudoers.d/` 아래 파일로 분리하는 것이 현대적인 방식입니다. 패키지 업데이트에 영향받지 않고, Ansible 같은 도구로 파일 단위 배포·회수가 쉽습니다. 파일명에 점(.)이나 ~ 가 들어가면 무시되니 90-deploy 처럼 짓습니다.

6.4 정책 문법

기본 형태는 다음과 같습니다.

```
사용자/그룹  호스트=(실행할_사용자:실행할_그룹)  명령목록
```

배포판 기본 설정에 있는 이 줄을 뜯어보면 이해가 빠릅니다.

```
%sudo  ALL=(ALL:ALL)  ALL
①      ②    ③    ④    ⑤
```

	필드	설명
①	%sudo	대상. % 가 붙으면 그룹, 없으면 사용자
②	ALL	호스트. 어떤 서버에서 적용할지. 보통 ALL
③	(ALL	어떤 사용자로 실행할 수 있는가
④	:ALL)	어떤 그룹으로 실행할 수 있는가
⑤	ALL	실행 가능한 명령 목록

6.5 실전 예시

특정 사용자에게 전권 부여

```
trinity ALL=(ALL) ALL
```

그룹 단위 부여

```
%developers ALL=(ALL) ALL
```

실행 가능한 명령을 제한 — 가장 실무적인 형태

```
trinity ALL=(ALL) /usr/bin/systemctl restart nginx, /usr/bin/journalctl
```

이제 trinity는 nginx 재시작과 로그 조회만 할 수 있습니다. 다른 명령을 시도하면 거부됩니다.

```
sudo rm -rf /var
# Sorry, user trinity is not allowed to execute '/usr/bin/rm -rf /var' as root
```

명령은 반드시 절대 경로로 적어야 합니다. systemctl 처럼 이름만 쓰면 PATH 조작으로 우회될 수 있습니다. which systemctl 로 경로를 확인해 적으세요.

다른 일반 사용자로 실행

```
trinity ALL=(deploy) /usr/bin/git
```

```
sudo -u deploy git pull
```

sudo는 root 전권이 아닙니다. -u 로 임의의 계정으로 명령을 실행할 수 있고, 정책의 세 번째 필드가 그 범위를 정합니다.

비밀번호 없이 실행

```
deploybot ALL=(ALL) NOPASSWD: /usr/bin/systemctl reload nginx
```

CI/CD 자동화 계정처럼 사람이 비밀번호를 입력할 수 없는 경우에 씁니다.

NOPASSWD: ALL 은 사실상 비밀번호 없는 root입니다. 그 계정의 SSH 키가 유출되면 즉시 서버 전체가 넘어갑니다. NOPASSWD를 쓸 때는 반드시 필요한 명령만 좁게 지정하세요.

6.6 권한 축소가 어려운 이유

명령을 제한해도 그 명령이 셸 탈출(shell escape)을 허용하면 무의미해집니다.

```
trinity ALL=(ALL) /usr/bin/vim # 위험!
```

vim 안에서 :!bash 를 치면 root 셸이 열립니다. less , more , find -exec , awk , tar --to-command 등도 같은 문제를 갖습니다. 파일 편집 권한을 줘야 한다면 sudoedit (sudo -e)를 쓰세요. 임시 복사본을 사용자 권한으로 편집한 뒤 되돌려 놓는 방식이라 안전합니다.

```
trinity ALL=(ALL) sudoedit /etc/nginx/nginx.conf
```

6.7 확인 명령

```
sudo -l          # 내가 실행할 수 있는 sudo 명령 목록
sudo -l -U trinity # 특정 사용자의 권한 확인
sudo -k          # 비밀번호 캐시 즉시 만료
```


7. root 계정 접근 통제

7.1 root가 되는 방법

명령	입력하는 비밀번호	조건
<code>sudo <명령></code>	내 비밀번호	sudo 권한 보유
<code>sudo -i</code> / <code>sudo --login</code>	내 비밀번호	sudo 권한 보유, root 로그인 셸로 전환
<code>su -</code> / <code>su -l</code>	root 비밀번호	root 비밀번호를 알고 있고 잠기지 않았을 것

root 세션에서 빠져나올 때는 `logout` 또는 `exit` 를 씁니다.

`su` 와 `su -` 의 차이도 자주 나옵니다. `su` 는 사용자만 바꾸고 **환경 변수는 이전 사용자 것을 유지합니다**. `su -` 는 로그인 셸을 새로 띄워 대상 사용자의 `PATH·HOME` 등을 제대로 적용합니다. 예상과 다른 `PATH` 때문에 명령을 못 찾는 문제를 피하려면 항상 `-` 를 붙이세요.

7.2 root 계정 잠금과 해제

많은 배포판(특히 Ubuntu)은 기본적으로 root 비밀번호 로그인을 잠가둡니다. 잠겨 있어도 `sudo -i` 는 `/` 비밀번호를 쓰므로 정상 동작합니다. 반면 `su -` 는 root 비밀번호를 요구하므로 실패합니다.

목적	명령
root 비밀번호가 없던 경우 새로 설정	<code>sudo passwd root</code>
과거에 설정했으나 잠긴 경우 해제	<code>sudo passwd -u root</code>
root 비밀번호 로그인 차단	<code>sudo passwd -l root</code>

root를 잠그기 전에 반드시 확인할 것. 현재 계정이 `sudo`를 쓸 수 있는지 먼저 검증하세요. root 로그인도 막고 `sudo` 권한도 없으면 관리 권한을 얻을 방법이 사라져 서버에서 잠깁니다. 복구하려면 물리 콘솔이나 클라우드 복구 모드가 필요합니다.

안전한 순서: ① 새 터미널에서 `sudo -v` 로 `sudo` 동작 확인 → ② root 잠금 → ③ 기존 세션은 그대로 두고 새 세션으로 재확인.

7.3 SSH 계층의 통제

계정 잠금과 SSH 접근 통제는 별개입니다. `passwd -l root` 는 비밀번호 로그인만 막을 뿐, root의 `authorized_keys` 에 공개키가 있으면 SSH 키 로그인도 여전히 가능합니다. SSH 레벨에서 확실히 막으려면 `/etc/ssh/sshd_config` 를 설정합니다.

```
PermitRootLogin no      # root SSH 로그인 완전 차단
PasswordAuthentication no  # 비밀번호 인증 비활성화, 키만 허용
```

```
sudo sshd -t            # 문법 검사 — 재시작 전에 반드시 실행
sudo systemctl reload sshd
```

표준 운영 구성: root SSH 직접 로그인 차단 → 개인 계정으로 키 인증 접속 → 필요할 때 `sudo` 로 권한 상승. 이렇게 하면 “누가 무엇을 했는지”가 개인 계정 단위로 로그에 남습니다. root 비밀번호를 팀에서 공유하는 방식은 감사 추적이 불가능해 지양됩니다.

8. 부록

8.1 배포판별 차이 정리

항목	Debian / Ubuntu	RHEL / Rocky / CentOS
관리자 그룹	sudo	wheel
계정 생성	adduser (대화형 래퍼)	useradd
계정 삭제	deluser	userdel
nologin 경로	/usr/sbin/nologin	/sbin/nologin
인증 로그	/var/log/auth.log	/var/log/secure
root 기본 상태	비밀번호 잠김	설치 시 비밀번호 지정
기본 umask	022	022 (UPG로 002인 경우도)

8.2 핵심 파일 요약

경로	내용	퍼미션
/etc/passwd	계정 기본 정보	644 (모두 읽기)
/etc/shadow	비밀번호 해시, 만료 정책	640 또는 000 (root만)
/etc/group	그룹 정보와 보조 멤버 목록	644
/etc/gshadow	그룹 비밀번호	root만
/etc/sudoers , /etc/sudoers.d/	sudo 정책	440
/etc/skel/	신규 계정 홈 템플릿	755
/etc/security/limits.conf , limits.d/	PAM 리소스 제한	644
/etc/environment	전역 환경 변수	644
/etc/profile.d/*.sh	전역 로그인 스크립트	644
/etc/login.defs	UID/GID 범위, 비밀번호 기본 정책	644

8.3 실습 체크리스트

VM이나 컨테이너에서 아래를 직접 해보면 이 문서 내용이 몸에 붙습니다. 각 항목은 검증 명령까지 포함해서 수행하세요.

- 계정을 만들고 `id` , `getent passwd` 로 UID/GID 확인
- `chage -d 0` 을 걸고 실제로 로그인 시 비밀번호 변경을 요구받는지 확인
- 그룹을 만들고 `usermod -aG` 로 추가 후 `groups` 로 검증
- 협업 디렉터리에 `setgid( chmod 2775 )`를 걸고, 걸기 전/후 파일 그룹 소유자가 어떻게 달라지는지 비교
- `usermod -G ( -a 없이)`를 **테스트 계정**에 실행해 보조 그룹이 날아가는 것을 직접 확인
- `/etc/profile.d/` 에 스크립트를 넣고 로그아웃/로그인으로 실행 확인
- `/etc/skel` 에 파일을 넣고 새 계정 생성 시 복사되는지 확인
- `limits.d` 로 `nproc` 을 3으로 제한하고 `ulimit -a` 및 `fork` 실패 확인
- `sudoers`에 명령 제한 정책을 넣고 `sudo -l` 로 확인 후 허용/차단 명령을 각각 실행
- 계정 잠금 후 **SSH 키로는 여전히 로그인되는지** 직접 확인 (1.8절 경고 검증)

8.4 면접 대비 질문

- 1. `useradd` 와 `adduser` 의 차이는? 자동화 스크립트에는 어느 쪽을 쓰겠는가?
- 2. 주 그룹과 보조 그룹의 차이를 새 파일 생성 관점에서 설명하라.
- 3. `usermod -G` 와 `usermod -aG` 의 차이로 생길 수 있는 사고는?
- 4. `/etc/passwd` 와 `/etc/shadow` 가 분리되어 있는 이유는?
- 5. 계정 만료와 비밀번호 만료의 차이는? 각각 어떤 상황에 쓰는가?
- 6. 퇴사자 계정을 처리하는 절차를 순서대로 설명하라. 왜 즉시 삭제하지 않는가?
- 7. `usermod -L` 만으로 접근을 완전히 차단할 수 있는가? 없다면 무엇이 더 필요한가?
- 8. 협업 디렉터리에서 `setgid` 비트가 필요한 이유는?
- 9. `/etc/profile` 을 직접 수정하지 않고 `/etc/profile.d/` 를 쓰는 이유는?
- 10. `~/.bashrc` 와 `~/.bash_profile` 은 각각 언제 읽히는가?
- 11. `limits.conf` 설정이 `systemd` 서비스에 적용되지 않는 이유와 해결 방법은?
- 12. `visudo` 를 써야 하는 이유는?
- 13. `sudo` 로 `vim` 실행을 허용하면 왜 위험한가? 대안은?
- 14. `docker` 그룹 소속이 `root` 권한과 사실상 동등한 이유는?
- 15. `root` 계정을 잠글 때 반드시 먼저 확인해야 할 것은?
- 16. `su` 와 `su -` 의 차이는?

8.5 자주 하는 실수 정리

실수	결과	올바른 방법
<code>usermod -G</code> (a 누락)	기존 보조 그룹 전부 소실	항상 <code>-aG</code>
<code>usermod -d</code> 만 사용	홈 경로만 바뀌고 파일은 옛 위치에 남음	<code>-d</code> 와 <code>-m</code> 함께
<code>/etc/sudoers</code> 직접 편집	문법 오류 시 <code>sudo</code> 전체 마비	<code>visudo</code> 사용
<code>sudoers</code> 에 상대 경로 명령	<code>PATH</code> 조작으로 우회 가능	절대 경로로 명시
<code>setgid</code> 없이 협업 디렉터리 운영	서로 파일을 못 읽음	<code>chmod 2775</code> + <code>umask 002</code>
잠금만으로 퇴사자 차단	SSH 키로 접속 가능	키 제거 + 셸 변경 + 계정 만료 병행
<code>limits.conf</code> 로 데몬 한도 조정 시도	적용 안 됨	<code>systemd</code> 유닛의 <code>LimitNOFILE</code>
SSH 설정 변경 후 세션 종료	오설정 시 재접속 불가	<code>sshd -t</code> 검사 + 기존 세션 유지

이 문서는 리눅스 사용자·그룹·권한 관리 주제를 학습용으로 새로 정리한 자료입니다. 명령 동작과 경로는 배포판·버전에 따라 달라질 수 있으므로, 운영 환경에 적용하기 전에 `man` 페이지와 해당 배포판 공식 문서로 확인하시기 바랍니다.